Tool-Using LLM Agents

Learning dependency structure and execution state instead of one fixed tool-call order

Research presentation based on the working draft QET v2 (Notion, 2026-10-13) and its earlier version (2026-10-08)

Use the arrow keys to move between slides and press L to switch between English and Chinese.

OVERVIEW

Outline and status labels

Part	Content
1 Motivation	Why one demonstrated order is a poor training target
2 Problem setup	Task input, environment snapshot, and demonstration
3 Execution graph	Dependencies, non-interference, and accepted executions
4 Execution state	Mapping equivalent histories to one model input
5 Shared-state computation	Dynamic programming over completed sets
6 Training objective	Marginal likelihood, gradients, and intermediate starts
7 Algorithms	Training and inference procedures
8 Positioning and status	Related work, assumptions, evidence, and open questions

Label	Meaning
From the v2 draft	Content of the reviewed draft
From the earlier draft	Definitions v2 uses but only the earlier draft states
Established result	Published or textbook result, checked at its source
Derived computation	Arithmetic on the drafts' definitions, computed by script
Illustrative numbers	Example values for teaching, not measurements
Hypothesis	A claim the drafts propose to test

Interpretations that go beyond both drafts carry the marker (my synthesis), used at most once per slide.

Method name: the drafts call the method QET; this deck uses the acronym only.

From the v2 draft

Running example: two loads and one comparison

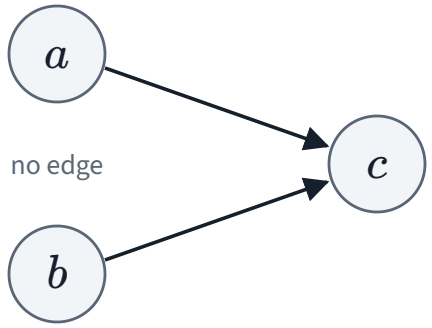
"Load dataset A and dataset B, then compare their statistics."

$$a = \mathrm{load}(\mathrm{A}), \qquad b = \mathrm{load}(\mathrm{B}), \qquad c = \mathrm{compare}\left(\mathrm{out}(a), \mathrm{out}(b)\right)$$

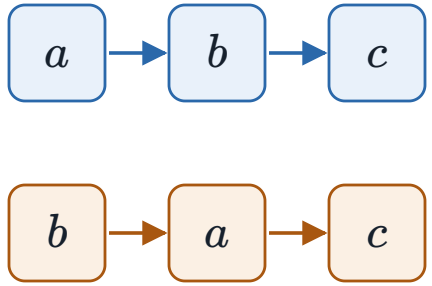
$$a \prec c, \qquad b \prec c$$

With fixed data snapshots and non-interfering loads, the only required orderings are a before c and b before c.

Nothing requires a to precede b, so (a, b, c) and (b, a, c) are two executions of the same dependency-structured computation.



dependency graph



two valid executions

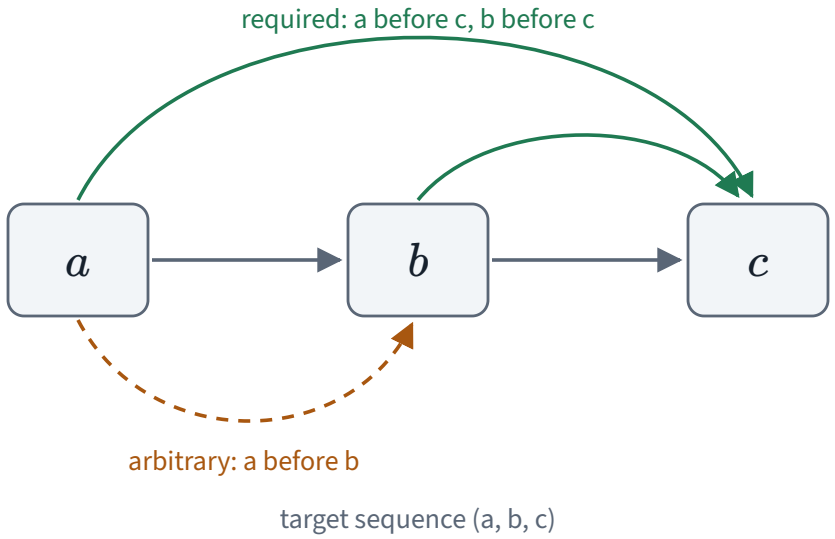
From the v2 draft

Task semantics and serialization choice

$$\mathcal{L}_{\mathrm{SFT}} = -\log P_\theta(a, b, c \mid x)$$

- **Task semantics.** They specify which actions and data dependencies are necessary, for example both loads must finish before the comparison runs.
- **Serialization choice.** It records which independent action the demonstrator happened to write first, here the load of A before the load of B.

The SFT target encodes both as the same relation, one span preceding another, so the loss cannot separate them.



From the v2 draft. For tool-using agents the problem is that the supervision signal may impose constraints that the task itself does not require.

From the v2 draft

Consequences for tool-using agents

Data and compute

One computation can admit many valid orders, so teaching each order as a separate example spends data and compute repeatedly on one computation.

Resuming partial work

An agent may need to continue work that another executor partly completed, and one memorized order from scratch does not guarantee this ability.

Argument availability

An agent must know which arguments are already available, which prerequisites are unmet, and which call produced each value.

From the v2 draft. The draft concludes that execution progress and argument provenance should become explicit objects of post-training.

Learning a correct call order is different from learning to manage execution state across several dependent tools.

Established result

Evidence from FuncBenchGen

FuncBenchGen builds tasks on a hidden function-dependency DAG, and the model must call functions until it can report a target value.

Failure type	Definition in the paper	Share of errors across 7 models
Value Not Yet Known	Uses a variable whose value no prior call or initial input has established	66.8% to 81.3%
Incorrect Value	Uses an established variable but with a wrong value	17.9% to 33.2%
Function Not Found; Wrong Number of Inputs	Schema-level errors	at most 2.4% and 0.9%

Most models write syntactically valid calls, yet they lose track of which values exist and which value belongs to which argument.

Restating all known variable values after every call raised the success rate of GPT-5 from 62.5% to 81.3%, according to the paper.

Maekawa et al., ICLR 2026 (arXiv 2509.26553): Table 3, Section 4.6, and the abstract.

From the v2 draft | Illustrative numbers

Two failure types in the running example

Value Not Yet Known

Completed A → flight_price = 1200

Missing B → hotel_price

```
calculate_total(flight_price=1200, hotel_price=?) ×
```

The agent called A but not B and already tries C, so it never checked the preconditions of C.

Incorrect Value

Completed A → 1200, B → 600

Passed hotel_price = 900

```
calculate_total(flight_price=1200, hotel_price=900) ×
```

The correct value 600 is already available, but the agent passes 900, possibly a value returned by another tool.

In both cases the link between a variable, the call that produced it, and its actual value has been lost.

Both errors concern execution state: which call is executable now, and which real arguments it should receive.

Hypothesis

Central research hypothesis

Hypothesis. An agent can learn the whole set of semantically valid tool-call orders and complete the remaining work from any valid intermediate state.

The earlier draft states a testable version for tasks whose ordering freedom is large and can be identified reliably.

Hypothesis. Under these conditions, training on the set of valid executions gives a better capability–compute trade-off than fixed-order supervision, order augmentation, or frontier supervision.

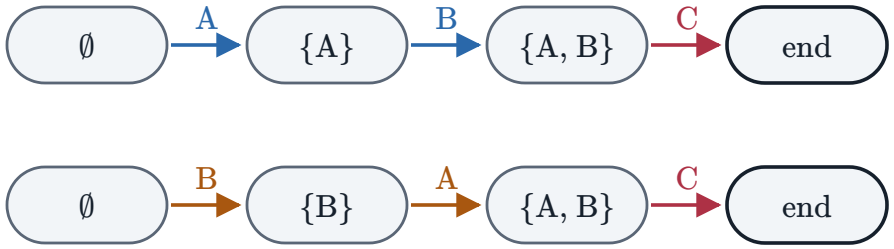
Neither draft reports training results, and the only numerical evidence is a self-check that the probability recursion and gradients are implemented consistently.

From the v2 draft

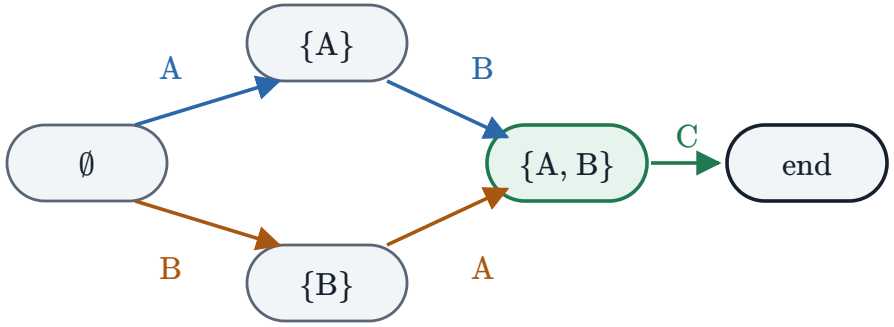
Limits of order augmentation

The direct fix samples more valid reorderings from the dependency graph and trains on each one as a separate sequence.

- **Sampling distribution.** Augmentation must choose a distribution over orders, and the model is then also trained to match that distribution.
- **Repeated scoring.** Trajectories that reach the same state after independent actions are still scored as separate sequences from start to end.
- **No resumption guarantee.** A model that reliably completes $A \to B \to C$ may still fail when another agent has already completed B.



per-path scoring: C is scored twice



shared state: C is scored once

From the v2 draft

Three changes to the training setup

	Standard SFT	Proposed method (QET)
Supervision target	One demonstrated trajectory τ	Set of accepted executions $\mathcal{C}(G)$
Policy input	Full chronological history h_t	Canonical execution state $H_t = \mathrm{Reduce}(h_t)$
Probability computation	Each path scored on its own	Dynamic programming over shared completed sets
Training start states	Task start only	Task start and valid intermediate states

The model learns a space of valid solutions instead of imitating one reference trajectory, and it learns to continue from partial progress.

v2 draft, solution items 1 to 3; the fourth row comes from the method section on valid prefixes.

From the v2 draft | From the earlier draft

Task input, environment, and demonstration

$$x = (u, \mathcal{T}, o_0) \qquad (s', o) = \mathrm{Exec}_\zeta(s, a) \qquad \tau = (a_1, o_1, \ldots, a_n, o_n, a^\star_{\mathrm{end}})$$

Symbol	Meaning	Running example
u	Natural-language request	Query flight and hotel prices from NYC to SG, then compute the total cost
$\mathcal{T}$	Tool descriptions	`get_flight_price`, `get_hotel_price`, `calculate_total`
o_0	Information the agent may use at the start	Not specified in the draft
ζ	Fixed environment snapshot: what is stored and how it is restored	Stored price data, restorable for every run (illustrative)
$s,\ s'$	Environment state before and after action a, held by the executor	–
o	Observation returned to the agent by this call	`flight_price = 1200`
$a^\star_{\mathrm{end}}$	Terminal action after successful completion	Return the total cost

The snapshot ζ belongs to the training executor and is never passed to the policy as privileged input.

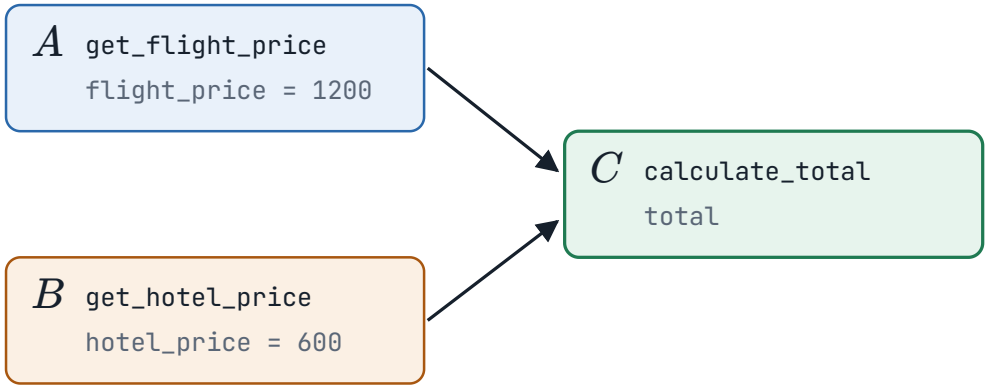
v2 draft, Section 2; the privileged-input rule is from the earlier draft, Section 3.1.

From the v2 draft

Target execution graph

$$G = (V, E, \mathcal{A}, \mathcal{O}, \mathcal{K})$$

Part	Meaning	Flight and hotel example
V	Operation instances the task must complete	$\{A, B, C\}$
E	Dependencies that must hold	$\{(A, C), (B, C)\}$
$\mathcal{A}$	Tool call and argument binding of each node	$a_C = \texttt{calculate_total}(p_f, p_h)$
$\mathcal{O}$	Fields that each call returns	$o_A : \{\texttt{flight_price}\}$
$\mathcal{K}$	Constraints on legality, data flow, and result checks	$\mathrm{CanExecute}(C) \Leftrightarrow \mathrm{Valid}(o_A) \wedge \mathrm{Valid}(o_B)$



E = {(A, C), (B, C)}; no edge between A and B

Nodes are operation instances rather than tool types, so calling the same tool twice creates two distinct nodes.

From the v2 draft · Established result

Operation record and non-interference

$$k_v = (a_v, R_v, W_v, P_v, O_v)$$

a_v	Executed action, such as `get_flight_price()`
R_v	Read set: database fields, files, and other state it reads
W_v	Write set: state it modifies
P_v	Argument sources and preconditions
O_v	Output returned by the tool

$$W_u \cap (R_v \cup W_v) = \varnothing, \qquad W_v \cap (R_u \cup W_u) = \varnothing$$

When both hold, neither operation writes anything the other reads or writes, so swapping them changes neither result.

Two pure queries have $W_u = W_v = \varnothing$ and always satisfy both conditions, so the flight and hotel lookups commute.

Correction. These are the Bernstein conditions (1966), which are sufficient for safe reordering but not necessary, and they assume complete read and write sets.

v2 draft, Section 3, states the condition as necessary; the earlier draft, Section 4.4, correctly calls it sufficient.

From the earlier draft | Illustrative numbers

Sources of dependency edges

Operation pair	Check	Result
`get_flight_price`, `get_hotel_price`	$W_u = W_v = \varnothing$	Commute, no edge
`update_budget(+100)`, `get_budget`	$W_u \cap R_v = \{\texttt{budget}\}$	Write–read conflict, keep the order
`lookup_address`, `send_message`	No write conflict, but sending needs the address as an argument	Data dependency, keep the order

- Non-interference alone does not decide independence, because a later call may need an earlier result as its argument.
- The earlier draft therefore builds E from data, guard, effect, and task edges, and keeps the demonstrated order for any unverified pair.
- All relevant pairs must be checked: if a writes x, b is unrelated, and c reads x, the edge $a \prec c$ must stay.

Earlier draft, Sections 4.3 to 4.5; the budget pair is an illustration added for this deck.

From the v2 draft

Linear extensions and accepted executions

$$\mathrm{LinExt}(G) = \{(A, B, C),\ (B, A, C)\}$$

$$\mathcal{C}(G) = \left\{(a_A, a_B, a_C, a^{\star}_{\mathrm{end}}),\ \ (a_B, a_A, a_C, a^{\star}_{\mathrm{end}})\right\}$$

- A linear extension of G orders all nodes so that each edge starts at an earlier node; $\mathrm{LinExt}(G)$ collects all of them.
- The accepted execution set $\mathcal{C}(G)$ writes each linear extension as full tool calls and appends the terminal action.
- Any generated sequence inside $\mathcal{C}(G)$ counts as a correct execution, even when it differs from the demonstration.
- Uncertain pairs keep their demonstrated order, so $\mathcal{C}(G)$ can be smaller than the set of all truly equivalent executions.

v2 draft, Section 3, which calls this set the execution acceptance class; last point from the earlier draft, Section 4.6.

From the v2 draft

From one target to a set of targets

$$\mathcal{L}_{\mathrm{SFT}} = -\log P_\theta(A, B, C \mid x)$$

$$\mathcal{L} = -\log \sum_{\tau \in \mathcal{C}(G)} P_\theta(\tau \mid x)$$

- Standard SFT raises the probability of the single demonstrated trajectory (A, B, C) and gives (B, A, C) no positive signal.
- The set objective gives a positive signal whenever the model places probability on any accepted execution.
- The model is no longer forced to follow the path that the demonstrator happened to fix in advance.

The objective moves from imitating one reference answer to learning a space of valid solutions.

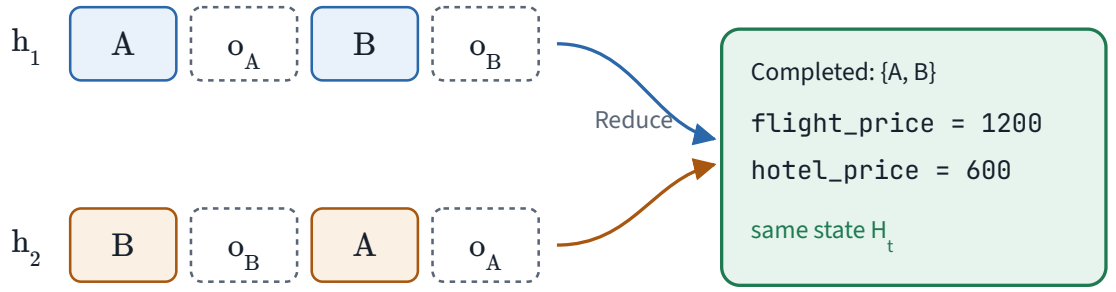
v2 draft, solution item 1. The draft labels the second trajectory with a repeated index; it is the second one.

From the v2 draft

History dependence of a Transformer policy

$$h_1 = (A, o_A, B, o_B), \qquad h_2 = (B, o_B, A, o_A)$$

- If A and B are independent and return the same results, both histories describe exactly the same task progress.
- A Transformer reads different token orders and positions, so it may still assign $P_\theta(C \mid h_1) \neq P_\theta(C \mid h_2)$.
- When next-action probabilities differ, the two histories cannot be merged into one state for exact probability computation.



different token orders, identical task progress

s_t	Environment state at step t, held by the executor
h_t	Full interaction history up to step t
H_t	Canonical observation state given to the LLM

v2 draft, Sections 1 and 3.1. v2 also uses s_t for the canonical state in its introduction; this deck uses H_t throughout.

From the v2 draft | From the earlier draft

Reduce and Render

$$\mathrm{Reduce}(h_1) = \mathrm{Reduce}(h_2) = H_t$$

$$\pi_\theta(a \mid x, H_t) = p_\theta\big(a \mid x, \mathrm{Render}(H_t)\big)$$

- Reduce maps every history with the same task progress to one canonical state built from completed actions and their observations.
- Render writes that state as fixed text, so both histories yield identical prompts and identical next-action distributions.
- Reduce may use only past actions, actual observations, and runtime contracts, never the target graph or future actions.

```
Completed operations:
- flight_query: price = 1200
- hotel_query: price = 600

Available values:
- flight_price: 1200, source = flight_query
- hotel_price: 600, source = hotel_query
```

Rendered state after either history, with the same text for h_1 and h_2.

From the earlier draft. Canonicalization drops only order certified as irrelevant, while overwrites of one variable and orders required by the task remain.

v2 draft, Section 3.1; the constraints on Reduce are from the earlier draft, Sections 5.2 and 5.3.

From the v2 draft | From the earlier draft

Probability of a complete action

$$\log \pi_\theta(a \mid x, H) = \sum_{\ell=1}^{L} \log p_\theta(a_\ell \mid x, H, a_{<\ell})$$

```
{
  "tool": "calculate_total",
  "arguments": {
    "flight_price": 1200,
    "hotel_price": 600
  }
}
```

The complete action a for node C, including its closing token.

- The action probability covers every token of the call: tool name, arguments, format, and the explicit end marker.
- A call with `hotel_price = 900` is a different token sequence, so the objective never rewards that wrong argument.
- Tool outputs enter as observations, not prediction targets, so deterministic accepted paths need no environment probability factor.

From the earlier draft. Scores use the actual policy probability, because renormalizing over the correct actions would discard the mass placed on wrong actions.

v2 draft, Section 3.1; the last two points are from the earlier draft, Sections 6.1 and 6.2.

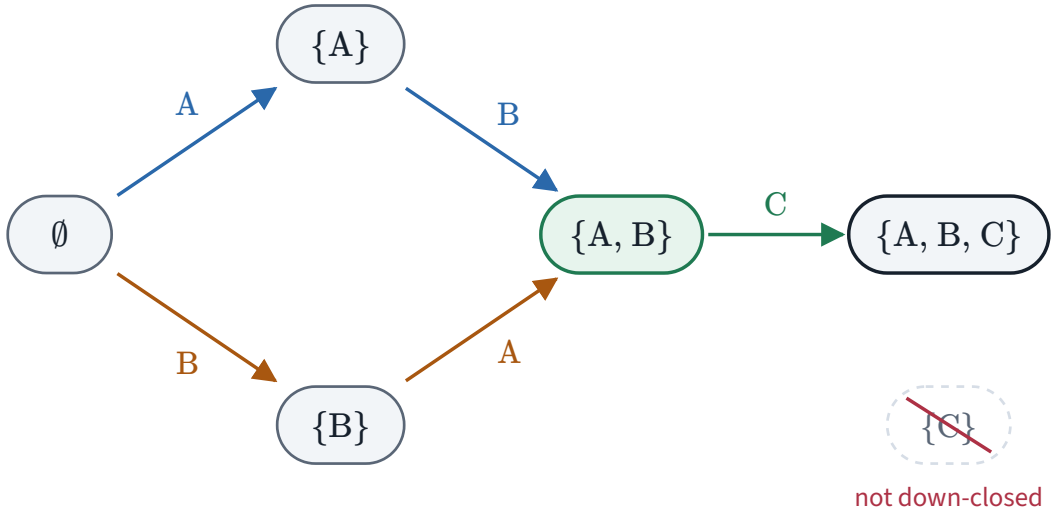
From the v2 draft · Established result

Completed sets and order ideals

$$v \in I \implies \mathrm{pa}_G(v) \subseteq I$$

$$\mathcal{J}(G) = \{\varnothing, \{A\}, \{B\}, \{A, B\}, \{A, B, C\}\}$$

- A completed set $I \subseteq V$ lists the operations already done, and it is valid only if it contains every prerequisite of its members.
- Such down-closed sets are called order ideals, and under inclusion they form a lattice that serves as the training state space.
- The set $\{C\}$ is invalid because C cannot be complete while its parents A and B are still missing.



valid completed sets $\mathcal{J}(G)$, ordered by inclusion

From the v2 draft | From the earlier draft

Condition for sharing a state

$$H_I = \mathrm{State}(x, I)$$

Merging is exact only when every accepted route to I yields the same canonical state, and therefore the same model input.

- **Deterministic accepted paths.** Tool results on accepted paths are deterministic, or they have a verified equivalent canonical form.
- **Order-invariant reduction.** Swapping two certified independent actions leaves the reduced state unchanged, written as $\mathrm{Reduce}(h\, a\, o_a\, b\, o_b) = \mathrm{Reduce}(h\, b\, o_b\, a\, o_a)$.
- **No hidden memory.** The policy carries no scratchpad across turns, so the same H_I always produces the same model input.

Note. Sharing saves computation only, since different routes to I keep their own probabilities, which are summed rather than assumed equal.

v2 draft, Section 3.1; the three conditions are from the earlier draft, Sections 5.2, 5.5, and 7.2.

From the v2 draft

Ready set and step scores

$$\mathcal{R}_G(I) = \{v \in V \setminus I : \ \mathrm{pa}_G(v) \subseteq I\}$$

$$w_\theta(I, v) = \pi_\theta(a_v \mid x, H_I), \qquad e_\theta(V) = \pi_\theta(a^\star_{\mathrm{end}} \mid x, H_V)$$

I	$\mathcal{R}_G(I)$	Scores needed
$\varnothing$	$\{A, B\}$	$w_\theta(\varnothing, A),\ w_\theta(\varnothing, B)$
$\{A\}$	$\{B\}$	$w_\theta(\{A\}, B)$
$\{B\}$	$\{A\}$	$w_\theta(\{B\}, A)$
$\{A, B\}$	$\{C\}$	$w_\theta(\{A, B\}, C)$
$\{A, B, C\}$	$\varnothing$	$e_\theta(V)$

- The ready set holds the unfinished operations whose prerequisites are all complete, so each one is executable now.
- $w_\theta(I, v)$ is the probability that the model emits exactly the call a_v at the state I.
- $e_\theta(V)$ is the probability of ending the task correctly once every operation is complete.
- This example needs 6 scores, one for each lattice edge plus one for the terminal action.

From the v2 draft | From the earlier draft

Completion probability recursion

$$B_\theta(I) = \sum_{v \in \mathcal{R}_G(I)} w_\theta(I, v)\, B_\theta(I \cup \{v\}), \qquad B_\theta(V) = e_\theta(V)$$

Term	Meaning
$B_\theta(I)$	Probability of finishing all remaining operations in some accepted order and then ending correctly, starting from I
$w_\theta(I, v)$	Probability of executing v next at state I
$B_\theta(I \cup \{v\})$	Completion probability after v is done
$B_\theta(V) = e_\theta(V)$	Base case: every operation is done and only the terminal action remains

Every accepted suffix begins with exactly one ready action, so summing over ready actions counts each suffix exactly once.

Correction. v2 defines $e_\theta(V)$ but leaves the base case implicit, and the earlier draft states $B_\theta(V) = e_\theta(V)$ explicitly.

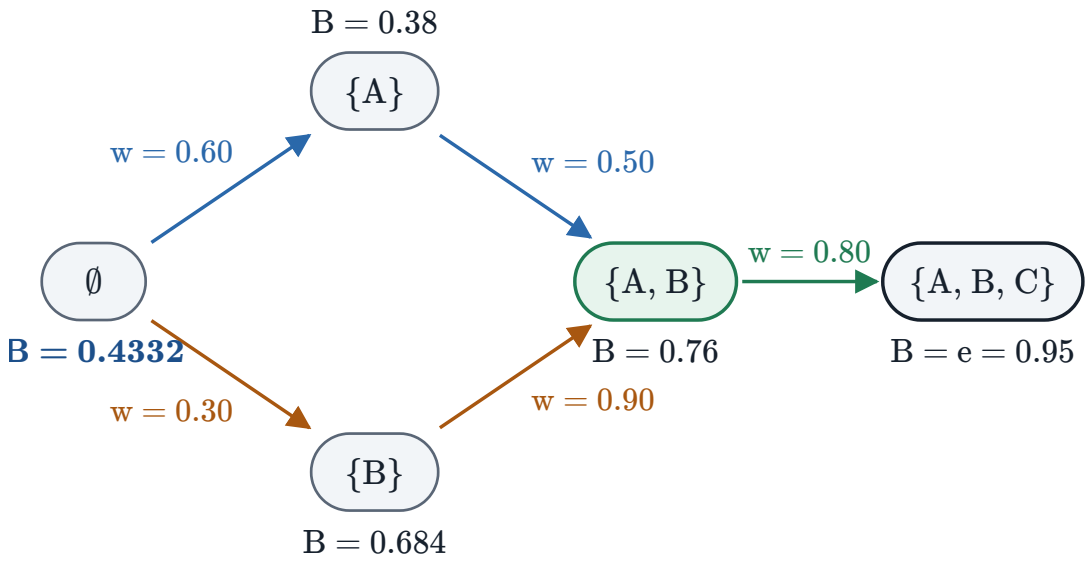
From the earlier draft | Illustrative numbers

Worked example, step by step

1. $B(V) = e(V) = 0.95$
2. $B(\{A, B\}) = 0.80 \times 0.95 = 0.76$
3. $B(\{A\}) = 0.50 \times 0.76 = 0.38$, and $B(\{B\}) = 0.90 \times 0.76 = 0.684$
4. $B(\varnothing) = 0.60 \times 0.38 + 0.30 \times 0.684 = 0.228 + 0.2052 = 0.4332$

The total equals the sum over both accepted paths, while the shared suffix through $\{A, B\}$ is scored only once.

At $\varnothing$ the remaining 0.10 of probability falls on actions outside the accepted set, such as calling C too early.



Policy scores from the earlier draft, Section 13.1; values computed by the build script; θ omitted.

From the v2 draft | Derived computation

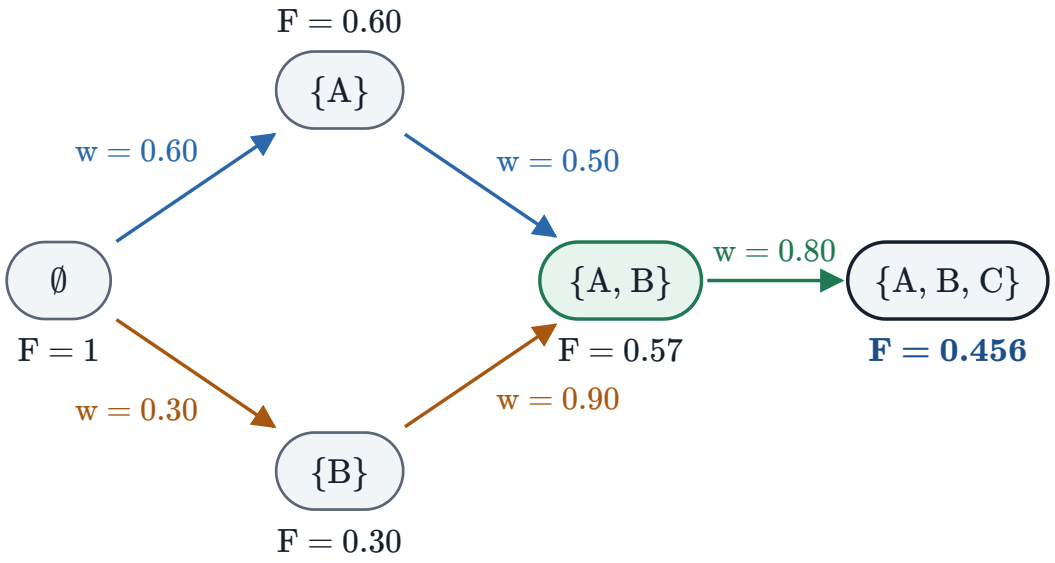
Forward view of the same sum

$$F(I \cup \{v\}) \mathrel{+}= F(I)\, w_\theta(I, v), \qquad F(\varnothing) = 1$$

1. $F(\{A\}) = 0.60, \quad F(\{B\}) = 0.30$
2. $F(\{A, B\}) = 0.60 \times 0.50 + 0.30 \times 0.90 = 0.30 + 0.27 = 0.57$
3. $F(V) = 0.57 \times 0.80 = 0.456$
4. $F(V)\, e_\theta(V) = 0.456 \times 0.95 = 0.4332 = B_\theta(\varnothing)$

$F(I)$ is the total probability of reaching I along accepted routes, and v2 introduces the method with this forward form.

Both directions give 0.4332, which is the forward–backward identity for path sums on a directed acyclic graph.



Correction. The v2 introduction omits the terminal action in F, and multiplying by $e_\theta(V)$ reconciles it with $B_\theta(\varnothing)$.

Derived computation | Established result

Why shared states matter

Dependency structure	Accepted orders	Ordered prefixes	Completed sets	Lattice edges
3 independent operations	6	16	8	12
10 independent operations	3,628,800	9,864,101	1,024	5,120
20 independent operations	2.43×10^{18}	6.61×10^{18}	1,048,576	10,485,760
4 chains of 5 operations	11,732,745,024	35,899,051,101	1,296	4,320
1 chain of 20 operations	1	21	21	20

- With n independent operations there are $n!$ orders but only 2^n completed sets, so 10 operations give 3,628,800 orders and 1,024 states.
- Ordered prefixes count the nodes of a prefix tree, so prefix caching alone cannot merge A then B with B then A (my synthesis).
- The state count can still grow exponentially, and the saving exists only when mergeable states actually occur, as v2 notes.

Counts computed by the build script; 1,296 matches the earlier draft, Section 10.1. Counting linear extensions is #P-complete (Brightwell and Winkler, 1991).

From the v2 draft | From the earlier draft

Marginal likelihood over accepted executions

$$\mathcal{L}_{\text{root}}(\theta) = \mathbb{E}_{(x,G)\sim\mathcal{D}}\big[-\log B_\theta(\varnothing)\big]$$

$$B_\theta(\varnothing) = P_\theta\big(\mathcal{C}(G)\big) = \sum_{\sigma\in\text{LinExt}(G)} p_\theta(\sigma)$$

- The root loss is the negative log probability that the agent completes the task through any accepted execution.
- In the worked example, the root loss for this single task equals $-\ln 0.4332 = 0.837$ nats.
- If every accepted execution passes the task check, $B_\theta(I)$ is a lower bound on the task success probability from I.
- The objective therefore targets a certified success event, not every strategy that might also solve the task.

v2 draft, class-marginal objective; the lower-bound remark is from the earlier draft, Section 7.4.

From the v2 draft | Derived computation

Three objectives on two example models

$$\mathcal{L}_{\text{SFT}} = -\log p_\theta(ABC)$$

$$\mathcal{L}_{\text{aug}} = -\tfrac{1}{2}\left[\log p_\theta(ABC) + \log p_\theta(BAC)\right]$$

$$\mathcal{L}_{\text{root}} = -\log\left[p_\theta(ABC) + p_\theta(BAC)\right]$$

Both models reach the completed state with total probability 0.50, so the marginal objective treats them as equivalent.

	Model 1	Model 2
$p_\theta(ABC)$	0.45	0.25
$p_\theta(BAC)$	0.05	0.25
Total over valid paths	0.50	0.50
$\mathcal{L}_{\text{root}}$	0.693	0.693
$\mathcal{L}_{\text{aug}}$	1.897	1.386
$\mathcal{L}_{\text{SFT}}$	0.799	1.386

- Order augmentation prefers Model 2 because it also rewards an even split between the two valid orders.
- SFT on (A, B, C) prefers Model 1, while the marginal objective has no preference between orders that are equally valid.

Probabilities and the first two loss rows from v2; the SFT row is derived for comparison. Losses in nats.

From the earlier draft · Established result

Decomposition of the augmentation loss

$$\mathbb{E}_{\sigma\sim\omega}\big[-\log p_\theta(\sigma)\big] = -\log P_\theta\big(\mathcal{C}(G)\big) + \mathcal{H}(\omega) + D_{\mathrm{KL}}\big(\omega \,\big\|\, p_\theta(\cdot \mid \mathcal{C}(G))\big)$$

- Training on orders drawn from ω equals the marginal loss, plus the entropy of ω, plus a term that fits ω.
- With a uniform ω over M orders, $\mathcal{H}(\omega) = \log M$ is constant, so only the KL term separates the two models.
- Averaging probabilities before the logarithm only adds $\log M$ to the marginal loss, so the gradient stays the same.

$$-\log\Big(\tfrac{1}{M}\sum_\sigma p_\theta(\sigma)\Big) = -\log\sum_\sigma p_\theta(\sigma) + \log M$$

	$\mathcal{L}_{\mathrm{aug}}$	$\mathcal{L}_{\mathrm{root}}$	$\mathcal{H}(\omega)$	D_{KL}
Model 1	1.897	0.693	0.693	0.511
Model 2	1.386	0.693	0.693	0.000

Each row satisfies $\mathcal{L}_{\mathrm{aug}} = \mathcal{L}_{\mathrm{root}} + \mathcal{H}(\omega) + D_{\mathrm{KL}}$ with uniform ω.

From the earlier draft. The extra KL term is not necessarily harmful and may act as a regularizer, so the comparison is an empirical question.

Earlier draft, Section 8.2, Eq. (21); v2 states the log-M identity without the decomposition.

From the earlier draft | Derived computation

Posterior-weighted gradient

$$\nabla_\theta \log \sum_{\sigma \in \mathcal{C}} p_\theta(\sigma) = \mathbb{E}_{\sigma \sim p_\theta(\cdot \mid \mathcal{C})} \left[\nabla_\theta \log p_\theta(\sigma) \right]$$

$$q_\theta(v \mid I, G) = \frac{w_\theta(I, v)\, B_\theta(I \cup \{v\})}{B_\theta(I)}$$

- The marginal gradient is an ordinary log-likelihood gradient, with each accepted path weighted by its posterior probability given success.
- In the worked example, the posteriors for the first action are $q(A \mid \varnothing) = 0.60 \times 0.38 / 0.4332 = 0.526$ and $q(B \mid \varnothing) = 0.474$.
- They differ from the policy's own share $0.60/0.90 = 0.667$, because completion after B is more likely.

Interpretation. Augmentation weights every order by the fixed ω, while the marginal gradient follows the success-conditioned preference of the model itself (my synthesis).

q_θ is defined in the earlier draft, Section 9.1; v2 uses it in Algorithm 1 without a definition.

From the earlier draft | Derived computation

Transition weights and the two-stage update

$$m_\theta(I) \leftarrow \mu_G(I), \qquad m_\theta(I \cup \{v\}) \mathrel{+}= m_\theta(I)\, q_\theta(v \mid I, G)$$

$$\rho_\theta(I, v) = m_\theta(I)\, q_\theta(v \mid I, G)$$

$$\widetilde{\mathcal{L}}_G = -\sum_{I,v} \operatorname{sg}\left[\rho_\theta(I, v)\right] \log w_\theta(I, v) - \log e_\theta(V)$$

- $\rho_\theta(I, v)$ is the expected number of times the success posterior takes action v at state I.
- The stop-gradient fixes the weights while token log probabilities are recomputed, which gives the exact marginal gradient at the current parameters.
- The value of this surrogate differs from the marginal loss, so training logs should report the dynamic-programming value.

Transition	ρ_θ
$(\varnothing, A)$	0.263
$(\varnothing, B)$	0.237
$(\{A\}, B)$	0.513
$(\{B\}, A)$	0.487
$(\{A, B\}, C)$	1.000
Sum	2.5

Start weights are $\mu(\varnothing) = 0.5$ and $\mu(\{A\}) = \mu(\{B\}) = 0.25$, as in the earlier draft, and the sum equals $\mathbb{E}[n - |S|]$.

Earlier draft, Sections 9.2 to 9.5 and 13.1; v2 Algorithm 1 computes these quantities without defining them.

From the v2 draft | Illustrative numbers

Limitation of root-only training

A policy that masters one order and rarely completes the task from other valid states can still reach a low root loss.

	Worked-example policy	Order-collapsed policy
$w(\varnothing, A),\ w(\varnothing, B)$	0.60, 0.30	0.85, 0.05
$w(\{A\}, B),\ w(\{B\}, A)$	0.50, 0.90	0.90, 0.10
Root loss $-\log B(\varnothing)$	0.837	0.536
Resume after B: $-\log B(\{B\})$	0.380	2.577

- The order-collapsed policy has the lower root loss, yet from $\{B\}$ it completes the task with probability only 0.076.
- A root-only objective cannot see this failure, because no training example starts after another agent has completed B.

Argument from v2 and the earlier draft, Section 8.3. The collapsed policy uses illustrative numbers (my synthesis); both share $w(\{A, B\}, C) = 0.80$ and $e(V) = 0.95$.

From the v2 draft

Training from intermediate states

$$\mu_G = (1-\lambda)\,\delta_{\varnothing} + \lambda\,\mu_{\text{cut}} \qquad \widehat{\mu}_G = (1-\lambda)\,\delta_{\varnothing} + \frac{\lambda}{K}\sum_{k=1}^{K}\delta_{I_k}$$

$$\mathcal{L}_{\text{QET}}(\theta) = \mathbb{E}_{(x,G)\sim\mathcal{D}}\,\mathbb{E}_{I\sim\mu_G}\big[-\log B_\theta(I)\big]$$

- Each update samples a task, then a valid start state I from μ_G, and trains completion from that state.
- λ sets the weight of non-root starts, and K sampled cut states replace enumeration of all valid completed sets.
- H_I is built only from the actual observations of actions in I, never from later steps of the demonstration.

Start I	Done	Remaining work
$\varnothing$	none	$A \to B \to C$ or $B \to A \to C$
$\{A\}$	A	$B \to C$
$\{B\}$	B	$A \to C$
$\{A, B\}$	A, B	C

v2 draft, valid-prefix coverage; the observation rule is from v2 Algorithm 1.

From the earlier draft | Illustrative numbers | Derived computation

Effect of intermediate starts

$$\mu(\varnothing) = 0.5, \qquad \mu(\{A\}) = \mu(\{B\}) = 0.25$$

	Worked-example policy	Order-collapsed policy
Root loss $-\log B(\varnothing)$	0.837	0.536
Mixture loss $\mathcal{L}_{\mathrm{QET}}$	0.755	1.007

$$0.5 \times 0.837 + 0.25 \times 0.968 + 0.25 \times 0.380 = 0.755$$
$$0.5 \times 0.536 + 0.25 \times 0.380 + 0.25 \times 2.577 = 1.007$$

- With a quarter of the weight on each one-action start, the worked-example policy now has the lower loss.
- If the actual resume distribution satisfies $\nu_G(I) \le \kappa\, \mu_G(I)$, the expected failure is bounded by $\mathbb{E}_{\nu_G}[1 - B_\theta(I)] \le \kappa\, \mathbb{E}_{\mu_G}[-\log B_\theta(I)]$.
- The bound follows from $1 - z \le -\log z$ and holds only for the covered tasks and valid resume states.

Start weights from the earlier draft, Section 13.1; coverage bound from its Proposition 3, Section 12.3. Losses in nats.

From the v2 draft

Training procedure (Algorithm 1)

Offline preparation

1. Record actions, argument sources, observations, state versions, and control conditions
2. Check all relevant action pairs and build a conservative target graph G
3. Verify unambiguous action events, demonstration success, and swap evidence
4. Split train, validation, and test sets by task and template

Each parameter update

1. Sample tasks with fixed graphs, then root and valid prefixes from μ_G
2. Build H_I from actual observations of completed actions
3. Small graph: full lattice. Large graph: bounded expansion
4. Compute B or a labeled lower bound B^-, then q, m, ρ
5. Fix posterior weights and recompute weighted token log probabilities
6. Accumulate gradients over micro-batches, then take one optimizer step

- Only the LLM or its parameter-efficient adapter is trained, while the graph compiler and the state interface stay fixed.
- All compared methods share format warm-up, tool descriptions, state templates, and comparable optimization, which isolates the effect of the objective.

From the earlier draft

Exact and bounded computation

$$B_\theta^-(I) \;\le\; B_\theta(I) \;\le\; B_\theta^+(I)$$

$$0 \;\le\; -\log B_\theta^-(I) + \log B_\theta(I) \;\le\; \log \frac{B_\theta^+(I)}{B_\theta^-(I)}$$

$$O(w\, N_G), \qquad N_G = |\mathcal{J}(G)| \le \prod_{j=1}^{k} (\ell_j + 1)$$

Note. v2 names B^-, B^+, C, and ε in Algorithm 1, but only the earlier draft defines them, in Section 10.

- Small graphs use the full lattice, and large graphs expand only part of it within the budget C.
- Unexpanded states take $B^- = 0$ and $B^+ = 1$, and the same recursion then yields guaranteed bounds on $B_\theta(I)$.
- Each sampled start is seeded with one complete accepted suffix, so B^- stays positive and the loss stays finite.
- When the budget runs out, training uses $-\log B^-$, an upper bound on the true loss, and reports it as such.
- Exact computation needs $O(w\, N_G)$ action scores, where w is the largest ready-set size and ℓ_j are the chain lengths of a chain decomposition of G.

Earlier draft, Sections 10.1 to 10.5.

From the v2 draft | From the earlier draft

Inference procedure (Algorithm 2)

Loop until the terminal action or the task budget

1. Initialize H_0
2. Generate a complete action a_t from x and $\mathrm{Render}(H_t)$
3. If a_t is the terminal action, output the result
4. Otherwise execute the call and receive the observation o_t
5. Update H_{t+1} with runtime information

- At inference the agent sees only x and $\mathrm{Render}(H_t)$, and it never accesses G, the lattice, B_θ, or q_θ.
- Training and inference share one implementation of Reduce and Render, so the model always sees the same state format.

Caveat. The class probability is defined under sampling, and greedy decoding can behave quite differently. Suppose ten correct first actions have probability 0.09 each and one wrong action has 0.10. Sampling then picks a correct first action with probability 0.90, but greedy decoding always picks the wrong one.

v2 draft, Algorithm 2; the access rule and the decoding example are from the earlier draft, Sections 11.2 and 11.3.

Established result | From the earlier draft

Related methods

Work	What it does	Difference stated in the earlier draft
Order-centric augmentation (He et al., 2025)	Generates valid reorderings from a dependency DAG as extra training data	Optimizes the marginal probability of the accepted set and covers intermediate states
De-linearizing agent traces (Li et al., 2026)	Infers latent partial orders from linear traces for faster execution	Uses a certified partial order as supervision to update the LLM policy
Order marginalization in graph generation (Chen et al., ICML 2021)	Treats node order as latent and trains a variational bound	Handles actions, observations, and executability in an interactive environment
Dynamic oracles (Goldberg and Nivre, TACL 2013)	Allow several correct transitions when training structured parsers	Scores whole accepted suffixes with exact state merging
LLMCompiler (Kim et al., ICML 2024) †	Plans and runs independent function calls in parallel at inference time	Changes the training objective rather than the executor

The earlier draft claims novelty only for the combination of certified equivalence, an online state interface, and completion likelihood with intermediate starts.

It explicitly does not claim to be the first to use partial orders, to sum over orders, or to allow several correct next steps.

The first four rows follow the earlier draft, Section 2.2; † marks a row added for this deck (my synthesis).

Established result

Foundations from concurrency and sequence modeling

Result	Statement	Role in this work
Bernstein conditions (1966)	Disjoint read and write sets let two program segments run in either order	Certifies which tool calls commute
Partial-order reduction (Godefroid, 1996)	Explores one representative of each class of equivalent interleavings	Same equivalence, used to define a training target instead of pruning a search
Order ideals and linear extensions (Brightwell and Winkler, 1991)	Counting linear extensions is #P-complete, while dynamic programming over ideals is exact	Explains why the lattice of completed sets is the state space
CTC (Graves et al., ICML 2006)	Sums over all alignments of one label sequence with forward–backward recursions	Same dynamic-programming pattern, with completed sets in place of alignments

The results in the first two columns are established, and the CTC analogy in the last row is an interpretation (my synthesis).

Sources listed on the references slide; the drafts cite partial-order methods (Godefroid) but not Bernstein or CTC.

From the earlier draft | From the v2 draft

Assumptions and scope

- **Deterministic accepted paths.** Accepted actions return fixed results under the snapshot, while invalid inputs may still produce random errors, as in FuncBenchGen.
- **Finite demonstrations.** Loops are unrolled to the number of iterations actually observed, and the graph does not cover other iteration counts.
- **Conservative graphs.** A wrong independence claim adds illegal executions to positive supervision, which the earlier draft names the highest-priority risk.
- **Successful prefixes only.** The objective covers legal partial executions and does not teach recovery after tool failures or wrong writes.
- **One accepted set per task.** Solutions that use other tools or algorithms lie outside $\mathcal{C}(G)$, so evaluation must use environment success.

Established result Hypothesis

Status of the claims

Established

- FuncBenchGen failure statistics and the reported gain from restating known variable values.
- Bernstein conditions as a sufficient test for safe reordering.
- Exactness of dynamic programming over order ideals for path sums.
- The marginal-likelihood gradient identity and the augmentation decomposition.

Derived in the drafts

- Correctness of the recursion under determinism, consistent states, and unambiguous actions.
- Bounds $B^- \leq B \leq B^+$ and the logarithmic gap bound.
- Coverage bound for resume distributions dominated by μ_G.
- Self-check on 30 random DAGs: differences of at most 5.6×10^{-17} and 2.2×10^{-16}.

Hypotheses, not yet tested

- H1: removing irrelevant order constraints helps learning under a limited budget.
- H2: training from intermediate states improves resumption.
- H3: shared states make marginal training affordable.
- H4: certification quality determines effectiveness.
- H5: gains grow with the real ordering freedom of tasks.

No LLM training or benchmark results exist yet, so every performance statement in this deck remains a hypothesis.

Earlier draft, Sections 12, 13.2, and 14.1; FuncBenchGen, Table 3 and Section 4.6.1.

From the earlier draft

Planned baselines and controls

Baseline	Definition	Controls for
Canonical-State SFT	Same state interface as the method, trained on one order	Gain from the state interface
Order-Augmented SFT	Dependency-preserving reorderings as extra sequences	Gain from seeing more orders
Prefix-Matched SFT	Same start prefixes as the method	Gain from extra state coverage
Frontier supervision	$-\log \sum_{v \in \mathcal{R}_G(I)} \pi_\theta(a_v \mid x, H_I)$	Need for marginalizing whole suffixes
Sampled-order marginalization	Probability sum over sampled, deduplicated accepted orders	Value of the shared-state algorithm

- All baselines receive the same variable table, because FuncBenchGen shows that restating known values alone changes success rates.
- Argument interfaces must also match across methods, either all handles or all literal values, otherwise the task itself changes.

Earlier draft, Sections 14.5 and 14.6.

From the earlier draft

Metrics and falsification criteria

Planned testbeds are FuncBenchGen extended with effect contracts and restorable snapshots, and an instrumented subset of AppWorld tasks.

Metrics

- Task success rate in the real environment, judged by a programmatic state check.
- Resume success from valid prefixes, with the backend restored to the matching state.
- Auxiliary errors: argument provenance, unknown-variable use, redundant calls, early termination, and side effects.
- Full cost: training tokens, GPU time, scoring calls, environment calls, and graph construction.

Falsification criteria

- Simple methods suffice: a canonical order or order augmentation matches the method at equal budget.
- Local supervision suffices: frontier supervision reaches the same result at much lower cost.
- Too little structure: most real task graphs reduce to chains, or certification costs too much.
- Representation dominates: gains come almost entirely from the variable table or canonical state.
- Decoding gap: likelihood gains do not turn into success under the deployed decoding strategy.

Earlier draft, Sections 14.2, 14.3, 14.7, and 14.10.

From the earlier draft | From the v2 draft

Open questions

- **Certification at scale.** Reliable read and write contracts for real tools are hard to obtain, and their coverage limits which tasks can enter training.
- **Choice of μ_{cut}.** v2 leaves the cut distribution unspecified, while the earlier draft samples a depth and then ready nodes up to it.
- **Posterior concentration.** If the model starts with one preferred order, the success posterior concentrates on it, and prefix coverage only partly counters this.
- **Cost of canonical states.** Rendered states need not share a text prefix, so reusing the key–value cache across states is not free.
- **Stochastic tools and several strategies.** Random tool outputs and multiple solution graphs need explicit outcome branches or unions of accepted sets.

Earlier draft, Sections 8.4, 15.1, 15.3, 15.5, 16.1, and 16.3.

Summary

- Sequence targets mix necessary dependencies with arbitrary order, so the proposed target is the set of accepted executions $\mathcal{C}(G)$.
- A canonical execution state turns equivalent histories into identical model inputs, which makes probability sharing across histories exact.
- Dynamic programming over completed sets computes the marginal likelihood, and its gradient weights each path by its success posterior.
- Training from valid intermediate states targets resumption, a failure mode that root-only training cannot observe.

Hypothesis. Whether these changes improve agents at a fixed budget is still untested, and the answer depends on reliable certification of independence.

APPENDIX

Notation

Symbol	Meaning
$x = (u, \mathcal{T}, o_0)$	Task input: request, tools, initial observation
$\zeta,\ \mathrm{Exec}_\zeta$	Environment snapshot and its executor
$\tau,\ a^\star_{\mathrm{end}}$	Demonstration and terminal action
$G = (V, E, \mathcal{A}, \mathcal{O}, \mathcal{K})$	Target execution graph
$k_v = (a_v, R_v, W_v, P_v, O_v)$	Operation record
$\mathrm{LinExt}(G),\ \mathcal{C}(G)$	Linear extensions; accepted executions
$h_t,\ H_t$	Raw history; canonical state
$\mathrm{Reduce},\ \mathrm{Render}$	History-to-state map; state-to-text map

Symbol	Meaning
$I,\ \mathcal{J}(G)$	Completed set; all order ideals
$\mathrm{pa}_G(v),\ \mathcal{R}_G(I)$	Parents of v; ready set at I
$w_\theta(I, v),\ e_\theta(V)$	Step score; terminal score
$B_\theta(I),\ F(I)$	Completion probability; forward probability
$q_\theta,\ m_\theta,\ \rho_\theta$	Success posterior; occupancy; transition weight
$\mu_G,\ \lambda,\ K$	Start distribution; non-root weight; sample count
$B_\theta^-,\ B_\theta^+,\ C,\ \varepsilon$	Lower and upper bounds; budget; tolerance
$\mathcal{L}_{\mathrm{root}},\ \mathcal{L}_{\mathrm{QET}}$	Root loss; loss with intermediate starts

Notation follows the v2 method section and the earlier draft, Section 3.3.

APPENDIX

References

1. Maekawa, S., Hassell, J., Pezeshkpour, P., Mitchell, T., Hruschka, E. Towards Reliable Benchmarking: A Contamination Free, Controllable Evaluation Framework for Multi-step LLM Function Calling. ICLR 2026. arXiv:2509.26553.
2. He, Q., He, Q., Liang, J., Xiao, Y., Zhou, W., Sun, Z., Yu, F. Order Doesn't Matter, But Reasoning Does: Training LLMs with Order-Centric Augmentation. arXiv:2502.19907, 2025.
3. Li, D., Cheng, Z., Nicholls, G. K., Kong, Q. De-Linearizing Agent Traces: Bayesian Inference of Latent Partial Orders for Efficient Execution. arXiv:2602.02806, 2026.
4. Chen, X., Han, X., Hu, J., Ruiz, F., Liu, L. Order Matters: Probabilistic Modeling of Node Sequence for Graph Generation. ICML 2021, PMLR 139:1630–1639.
5. Goldberg, Y., Nivre, J. Training Deterministic Parsers with Non-Deterministic Oracles. TACL 1:403–414, 2013.
6. Kim, S., Moon, S., Tabrizi, R., Lee, N., Mahoney, M., Keutzer, K., Gholami, A. An LLM Compiler for Parallel Function Calling. ICML 2024, PMLR 235:24370–24391.
7. Graves, A., Fernández, S., Gomez, F., Schmidhuber, J. Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks. ICML 2006, 369–376.
8. Godefroid, P. Partial-Order Methods for the Verification of Concurrent Systems. LNCS 1032, Springer, 1996.
9. Bernstein, A. J. Analysis of Programs for Parallel Processing. IEEE Transactions on Electronic Computers EC-15(5):757–763, 1966.
10. Brightwell, G., Winkler, P. Counting Linear Extensions. Order 8(3):225–242, 1991.
11. Trivedi, H., Khot, T., Hartmann, M., Manku, R., et al. AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents. ACL 2024, 16022–16076.